

Lab 6 补充：从内核恒等映射升级到独立用...

[!IMPORTANT]

本章节是对现有 Lab 6 手册的补充。在 Lab 5 中，你使用了简化方案——将用户代码映射到内核恒等映射页表中，自始至终只用一张页表。本章节将引导你升级为**独立用户页表**，实现真正的用户/内核地址空间隔离。这是 xv6 原版的设计，也是现代操作系统的标准做法。

阅读方式提示：本章节不会直接给出可复制的完整代码。它会讲解原理、提供关键思路和伪代码，然后让你自己动手实现。建议边读边写代码，遇到思考题时先自己想清楚再继续。

第〇节：为什么需要独立用户页表？

0.1 Lab 5 简化方案的局限

在 Lab 5 中，你的 `userinit()` 直接在内核页表中用 `mappages(..., PTE_U)` 为用户代码建立映射。这意味着：

- 用户程序和内核共享同一张页表（只有 `PTE_U` 位的区别）
- 用户程序理论上能“看到”所有内核地址空间的布局（虽然没有 `PTE_U` 无法访问）
- 无法实现进程间地址空间隔离（多个用户进程的地址空间互相可见）
- `fork()` 时无法简洁地复制用户地址空间

0.2 独立用户页表的目标

升级后，每个用户进程拥有自己独立的页表：

1 内核页表 (kernel_pagetable)		用户页表 (p->pagetable)
2		
3 VA:0x80000000 → 内核代码		VA:0x0 → 用户代码页
4 VA:0x80100000 → 内核数据		VA:0x1000 → 用户栈页
5 VA:UART0 → UART设备		(仅包含用户自己的映射)
6 VA:TRAMPOLINE → 蹦床代码	← 相同 →	VA:TRAMPOLINE → 蹦床代码
7 (整个物理内存的恒等映射)		VA:TRAPFRAME → 陷阱帧
8		

关键变化：进入用户态前，内核要把 `satp` 切换到用户页表；从用户态返回内核时，要把 `satp` 切回内核页表。

第一节：核心难题——页表切换的“鸡生蛋”困境

1.1 问题的本质

在 `usertrapret()` 中，你需要做这件事：

```
1 w_satp(MAKE_SATP(p->pagetable)); // 切换到用户页表
```

```

2 sfence_vma(); // 刷新 TLB
3 asm volatile("sret"); // 进入用户态

```

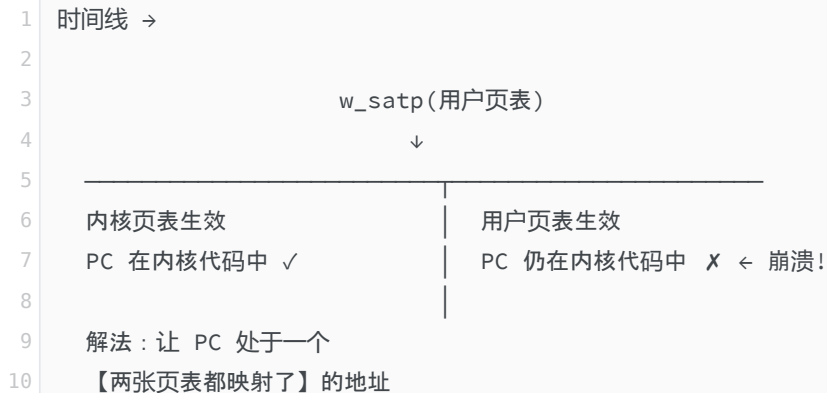
致命问题： `w_satp()` 执行后，MMU 立即开始使用新的用户页表翻译所有地址。但此时 CPU 的 PC 仍然指向 `usertrapret()` 函数体内的下一条指令——这条指令位于内核 `.text` 段（物理地址如 `0x8020xxxx`）。

如果用户页表没有映射这个内核代码地址（正常情况下不应该映射），那么 `w_satp()` 之后的第一条指令就会触发 **Instruction Page Fault**——系统崩溃。

同样的问题也出现在反方向：用户态 `ecall` 后，`stvec` 指向的处理函数（如 `usertrap()`）的地址只存在于内核页表中。但此时 `satp` 仍然是用户页表——CPU 跳转过去同样会 page fault。

🤔 **思考题1：** 你能想到哪些方案来解决这个困境？（提示：既然问题是“切换页表后 PC 指向的代码找不到了”，那核心思路是什么？）

1.2 问题的可视化



1.3 解法：Trampoline（蹦床页）

xv6 的方案：找一段“桥梁代码”，让它在两张页表中都被映射到完全相同的虚拟地址。这样，无论当前活跃的是哪张页表，CPU 都能找到这段代码继续执行。

```

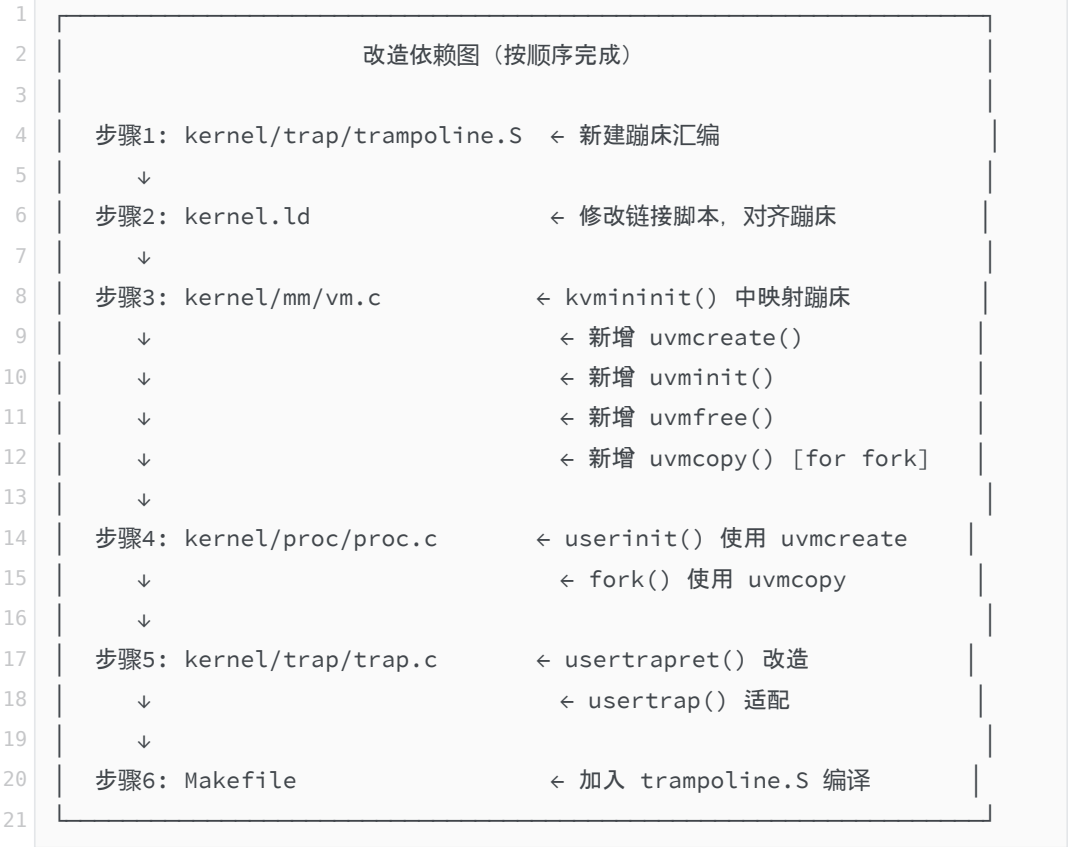
1 虚拟地址 TRAMPOLINE (= MAXVA - PGSIZE)
2  = 0x3FFFFFF000 (Sv39 最高可用虚拟页)
3
4
5 内核页表： VA:TRAMPOLINE → PA:trampoline代码
6 用户页表： VA:TRAMPOLINE → PA:trampoline代码
7
8 → 同一虚拟地址，同一物理页
9 → 切换 satp 前后，PC 始终有效
10

```

🤔 **思考题2：** 为什么蹦床页选择放在虚拟地址空间的最高处（`MAXVA - PGSIZE`）？放在别的位置可以吗？放在低地址可能有什么问题？

第二节：总体改造蓝图

在开始任何代码之前，先理解你需要改动的所有位置和它们之间的依赖关系：



第三节：步骤1——编写 trampoline.S

3.1 蹦床代码的职责

trampoline.S 包含两段代码，它们是”桥梁”：

入口标签	调用方向	职责
uservec	U-Mode → S-Mode	保存用户寄存器到 trapframe，切换到内核页表和内核栈，跳入 usertrap()
userret	S-Mode → U-Mode	切换到用户页表，从 trapframe 恢复用户寄存器，执行 sret

3.2 uservec 的核心问题

当用户态发生陷阱时，硬件只做三件事：`sepc ← PC`、`scause ← 原因`、`PC ← stvec`。硬件不保存任何通用寄存器，也不切换栈。因此 `uservec` 必须在不破坏任何用户寄存器的前提下完成保存工作。

[!CAUTION]核心约束：进入 `uservec` 时，所有通用寄存器仍然是用户程序的值。你不能随意使用任何寄存器作为临时变量，否则会丢失用户数据！

🤔 思考题3：你不能使用任何通用寄存器，但你需要知道 `trapframe` 在内存中的地址才能执行 `sd` 指令保存寄存器。你打算从哪里获取 `trapframe` 的地址？（提示：RISC-V 有哪些特殊的 CSR 寄存器可以在不影响通用寄存器的情况下读写？）

xv6 的解法是利用 RISC-V 的 `sscratch` 寄存器作为“逃生通道”：

```
1 # 进入 uservec 时的寄存器状态：
2 #   所有 x0-x31 = 用户程序的值（不能碰！）
3 #   sscratch = trapframe 的虚拟地址（由 usertrapret 预先设置）
```

3.3 `uservec` 的逻辑流程

请根据以下逻辑流程自行编写 `uservec` 的汇编代码：

```
1 第1步：原子交换 a0 和 sscratch
2   → 使用 csrrw 指令
3   → 效果：a0 获得 trapframe 地址（可用于寻址），用户原始 a0 存入 sscratch
4   → 想一想：为什么选择 a0 而不是其他寄存器？
5
6 第2步：以 a0 为基地址，用 sd 指令将所有用户寄存器保存到 trapframe
7   → 偏移量必须与 struct trapframe 中的字段顺序严格对应
8   → 注意跳过 a0 本身（此时它的值已在 sscratch 中）
9
10 第3步：从 sscratch 取回用户原始 a0，保存到 trapframe 对应位置
11   → 使用 csrr 读取 sscratch
12
13 第4步：从 trapframe 加载内核信息
14   → kernel_satp, kernel_sp, kernel_trap, kernel_hartid
15   → 这些字段在 struct trapframe 的前 5 个位置
16
17 第5步：切换到内核页表
18   → csrw satp + sfence.vma
19   → 此刻 PC 仍在 TRAMPOLINE 地址，安全！
20
21 第6步：跳入 usertrap()
22   → 使用 jr 指令
```

3.4 `userret` 的逻辑流程

`userret` 是 `uservec` 的镜像操作。它由 `usertrapret()` 通过函数指针调用，接收两个参数：

参数	含义
<code>a0</code>	<code>TRAPFRAME</code> 的虚拟地址（用户页表中 <code>trapframe</code> 的映射地址）
<code>a1</code>	用户页表的 <code>satp</code> 值

请根据以下逻辑自行编写：

- 1 第1步：切换到用户页表（`csrw satp + sfence.vma`）
- 2 第2步：以 `a0` 为基地址，从 `trapframe` 恢复所有用户寄存器
→ 注意：`a0` 必须最后恢复（因为恢复后 `trapframe` 指针就丢了）
- 4 第3步：执行 `sret` 返回用户态

3.5 关键：偏移量计算

[!WARNING]偏移量对照方法：打开你的 `proc.h`，找到 `struct trapframe` 的定义。
每个 `uint64` 字段占 8 字节。按声明顺序计算偏移：

字段位置	字段名	偏移（字节）
第 1 个	<code>kernel_satp</code>	0
第 2 个	<code>kernel_sp</code>	8
第 3 个	<code>kernel_trap</code>	16
第 4 个	<code>epc</code>	24
第 5 个	<code>kernel_hartid</code>	32
第 6 个	<code>ra</code>	40
第 7 个	<code>sp</code>	48
...	...	...

请自己继续完成后面所有寄存器的偏移量计算。**特别注意** `a0` 在第几个位置——
`uservec` 中跳过 `a0`、`userret` 中最后恢复 `a0` 都需要这个偏移量。
如果你的 `struct trapframe` 字段顺序与框架不同，所有偏移量都必须相应调整，否则寄存器会“张冠李戴”！

3.6 汇编框架提示

创建文件 `kernel/trap/trampoline.S` 时，需要注意以下结构要点：

```

1      .section trampsec          # 放入专用段（链接脚本需要对应处理）
2      .globl trampoline
3  trampoline:
4
5      .align 4
6      .globl uservec
7  uservec:
8      # TODO: 在此实现 uservec 逻辑（参照 3.3 节的流程）
9      # 提示：约 30~35 条汇编指令
10
11     .globl userret
12  userret:
13     # TODO: 在此实现 userret 逻辑（参照 3.4 节的流程）
14     # 提示：约 30~35 条汇编指令

```

🤔 思考题4：为什么 `trampoline.S` 使用了 `.section trampsec` 而不是默认的 `.text`？这与链接脚本有什么关系？

第四节：步骤2——修改链接脚本 `kernel.ld`

4.1 为什么需要修改？

蹦床代码必须满足两个条件：

1. 恰好占据一整个物理页（4096 字节），不能跨页
2. 页对齐：起始地址必须是 4096 的整数倍

这样内核才能把“蹦床代码所在的那一整页”同时映射到内核页表 and 用户页表的 `TRAMPOLINE` 虚拟地址。

4.2 修改要点

在 `kernel.ld` 的 `SECTIONS` 中，在 `.text` 段之后、`.rodata` 之前，添加一个新段：

- 使用 `. = ALIGN(0x1000)` 确保页对齐
- 为新段取名（例如 `.trampsec`），让链接器将 `trampoline.S` 中标记为 `trampsec` 段的代码收入其中
- 使用 `PROVIDE(trampoline = .)` 导出一个符号，使 C 代码能通过 `extern char trampoline[]` 获取蹦床代码的物理地址

🤔 思考题5：为什么需要在蹦床段的前后都加 `ALIGN(0x1000)`？只在前面加一个够不够？（提示：考虑紧接其后的 `.rodata` 段会不会侵入蹦床页。）

4.3 验证方法

编译后用以下命令确认蹦床页的位置：

```
1 riscv64-unknown-elf-objdump -h kernel.elf | grep trampsec
```

应看到该段的起始地址是 `0x1000` 对齐的，大小不超过 `0x1000` 。

第五节：步骤3——扩展 `vm.c`

5.1 在 `kvmininit()` 中映射蹦床页

在内核页表初始化的最后，你需要添加一条映射：

- 物理地址： `trampoline` （链接脚本导出的符号）
- 虚拟地址： `TRAMPOLINE` （ `memlayout.h` 中定义为 `MAXVA - PGSIZE` ）
- 权限： `PTE_R | PTE_X` （不带 `PTE_U` ）

[!IMPORTANT]为什么蹦床页不设 `PTE_U` ？

虽然 `stvec` 指向的蹦床地址在用户页表中也有映射，但用户程序**不应该**能主动跳到蹦床代码执行。蹦床入口只在硬件陷阱时被自动跳转到——RISC-V 规定，硬件陷阱跳转到 `stvec` 时不检查 `PTE_U` 位（因为此时 CPU 已经处于 S-Mode）。

[!WARNING]注意 `mappages` 参数顺序：本框架 `vm.c` 中 `mappages` 的实际参数顺序是 `(pagetable, pa, va, size, perm)`，而 `defs.h` 中的声明可能与此不同。请以 `vm.c` 中的实际实现为准，如果二者不一致，需要统一修正。

5.2 新增 `uvmcreate()` 一 创建用户页表

这个函数为一个新进程创建一张空白的用户页表。你需要在其中完成以下步骤：

1. 分配一页物理内存作为根页表 → `kalloc()`
2. 清零（否则随机数据会被当成有效 PTE）→ `memset()`
3. 映射蹦床页（与内核页表指向同一物理页）
 `VA:TRAMPOLINE → PA:trampoline`
 权限：`PTE_R | PTE_X`
4. 映射 `trapframe` 页
 `VA:TRAPFRAME → PA: (参数传入的 trapframe 物理地址)`
 权限：`PTE_R | PTE_W`
5. 返回新页表，失败返回 0

🤔 思考题6：

•

`TRAPFRAME` 的虚拟地址是什么？打开 `memlayout.h` 查看它和 `TRAMPOLINE` 的关系。

•

为什么 `trapframe` 需要 `PTE_W`（可写）权限？谁会往里面写数据？

•

为什么 `trapframe` 不设 `PTE_U`？（提示：谁在什么特权级下访问 `trapframe`？）

5.3 新增 `uvminit()` — 将用户代码装入用户页表

这个函数负责将用户代码（如 `proczero_code[]`）加载到用户页表的虚拟地址 0 处。

你需要：

1. 分配一页物理内存
2. 清零并拷贝用户代码
3. 在用户页表中建立映射：`VA:0 → 新分配的物理页`，权限 `PTE_R | PTE_W | PTE_X | PTE_U`

🤔 思考题7：为什么这里需要 `PTE_U` 权限，而蹦床页不需要？

5.4 新增 `uvmcopy()` — 为 `fork()` 复制用户地址空间

这个函数接收父进程页表、子进程页表 and 用户地址空间大小，逐页复制用户内存。

伪代码：

```
1 uvmcopy(old_pagetable, new_pagetable, size):
2     for 每一页虚拟地址 va (从 0 到 size, 步长 PGSIZE):
3         1. 在 old_pagetable 中用 walk() 找到 va 对应的 PTE
4         2. 从 PTE 中提取物理地址 pa 和权限 flags
5         3. 分配一页新物理内存
6         4. 拷贝 old PA 的内容到新物理页
7         5. 在 new_pagetable 中建立映射：同样的 VA → 新 PA, 相同权限
8     如果任何步骤失败, 返回 -1
9     全部成功, 返回 0
```

🤔 思考题8：`uvmcopy` 只复制用户代码/数据页。那蹦床页和 `trapframe` 页呢？它们是在 `uvmcopy` 中处理，还是在别的地方？（提示：看看 `uvmcreate()` 做了什么。）

5.5 工具函数：`walkaddr()`

如果你的框架中 `walkaddr()` 尚未实现，需要补充。这个函数在给定页表中查找虚拟地址对应的物理地址，只返回用户可访问（`PTE_U`）的有效页地址。

它的逻辑很简单：调用 `walk()` 拿到 PTE，检查 `PTE_V` 和 `PTE_U` 位，然后用 `PTE2PA()` 提取物理地址。

第六节：步骤4——改造 `userinit()` 和 `fork()`

6.1 `userinit()` 的改造

回顾 Lab 5 中 `userinit()` 的关键步骤，然后对照思考需要哪些变化：

步骤	Lab 5 简化版	独立用户页表版（你需要改成什么？）

页表选择	在内核页表中映射	?
用户代码虚拟地址	等于物理地址（恒等映射）	?
<code>trapframe->epc</code>	物理地址	?
页表切换	不需要	?
蹦床页	不需要	?

🤔 思考题9： 请填完上面表格右列的问号。提示：

- 用户代码应该从虚拟地址几开始执行？
- `trapframe->epc` 应该存虚拟地址还是物理地址？
- 需要调用你在 5.2 和 5.3 节实现的哪两个函数？

6.2 `fork()` 中使用 `uvmcopy`

在 `fork()` 中，为子进程创建用户地址空间的流程是：

1. 调用 `allocproc()` 获取子进程 PCB
2. 为子进程创建用户页表 → 调用你实现的 `uvmcreate()`
3. 复制父进程的用户页 → 调用你实现的 `uvmcopy()`
4. 复制 `trapframe` 内容 → `memmove`
5. 设置子进程的 `fork` 返回值 = 0 → `trapframe->a0 = 0`

🤔 思考题10： 如果 `uvmcopy()` 在复制到第 3 页时 `kalloc()` 返回 0（内存不足），你应该怎么办？需要释放哪些资源？

第七节：步骤5——改造 `usertrapret()` 和 `usertrap()`

这是整个改造中最关键的步骤。

7.1 `usertrapret()` 需要做什么？

对比 Lab 5 中你的 `usertrapret()` 和升级后需要做的事：

Lab 5 的做法	需要改成什么？	为什么？
-----------	---------	------

<code>w_stvec((uint64)usertrap)</code>	指向蹦床中的 <code>uservec</code>	下次陷阱时 <code>satp</code> 是用户页表， <code>usertrap</code> 的内核地址查不到
不填 <code>trapframe->kernel_satp</code>	填入当前内核页表	<code>uservec</code> 需要用它切回内核页表
不设 <code>sscratch</code>	设为 <code>TRAPFRAME</code>	<code>uservec</code> 入口处需要从 <code>sscratch</code> 获取 <code>trapframe</code> 地址
<code>asm volatile("sret")</code>	跳入蹦床的 <code>userret</code> 函数	需要在蹦床代码中切换页表 后再 <code>sret</code>

[!IMPORTANT]关键地址计算：

`stvec` 必须指向蹦床页中 `uservec` 的虚拟地址。但 `uservec` 这个 C 符号给出的是蹦床代码的物理地址。你需要计算：

```
1 uservec 在蹦床内的偏移 = (uint64)uservec - (uint64)trampoline
2 stvec 应设为 = TRAMPOLINE + 上述偏移
```

`userret` 的虚拟地址也用同样的方法计算。

你需要在文件顶部声明这些外部符号（来自 `trampoline.S` 和链接脚本）：

```
1 extern char trampoline[]; // 蹦床代码的物理地址
2 extern char uservec[];    // uservec 标签的物理地址
3 extern char userret[];    // userret 标签的物理地址
```

`usertrapret()` 的完整流程（请自行实现）：

```
1 1. intr_off() // 关中断
2 2. 计算并设置 stvec = TRAMPOLINE + uservec 偏移 // 蹦床入口
3 3. 填充 trapframe 的内核信息：
4   - kernel_satp = r_satp() // 当前内核页表
5   - kernel_sp = p->kstack + PGSIZE // 内核栈顶
6   - kernel_trap = (uint64)usertrap // C 处理函数
7   - kernel_hartid = r_tp() // CPU ID
8 4. 配置 sstatus：SPP=0, SPIE=1
9 5. w_sepc(trapframe->epc) // 用户程序 PC
10 6. w_sscratch(TRAPFRAME) // 预设 sscratch
11 7. 计算用户页表的 satp 值
```

- 12 8. 计算 `userret` 的虚拟地址 = `TRAMPOLINE` + `userret` 偏移
- 13 9. 通过函数指针调用 `userret(Trapframe, satp)`

🤔 思考题11：为什么第 9 步是调用函数指针而不是直接 `asm("sret")`？如果你直接在 `usertrapret()` 中切换 `satp` 然后 `sret`，会发生什么？

7.2 `sscratch` 寄存器的作用

如果你的 `riscv.h` 中没有 `sscratch` 的读写函数，需要自行添加。可以参考 `riscv.h` 中已有的其他 CSR 读写函数的写法（如 `r_satp()` / `w_satp()`）。

`sscratch` 的工作时序：

```
1  usertrapret():
2      w_sscratch(Trapframe)    ← 预设 sscratch = trapframe 虚拟地址
3      → userret → sret → 用户态
4
5  用户程序执行中...
6      sscratch 的值一直是 Trapframe（用户程序不会碰 sscratch）
7
8  ecall / 中断发生：
9      PC ← stvec（蹦床 uservec）
10     uservec：
11         csrrw a0, sscratch, a0
12         → a0 获得 Trapframe 地址，可以开始保存寄存器了
13         → 用户原始 a0 存入 sscratch
```

7.3 `usertrap()` 的适配

`usertrap()` 本身不需要大改，但需要注意一个新增的步骤：

由于蹦床版的 `uservec` 已经把所有用户寄存器保存到了 `trapframe`，但 `sepc` 的值还在 CSR 中，没有自动存到 `trapframe`。你需要在 `usertrap()` 的开头手动执行：

```
1  p->trapframe->epc = r_sepc();
```

🤔 思考题12：为什么需要把 `sepc` 保存到 `trapframe`？如果不保存，在什么情况下会出错？（提示：`usertrap()` 处理过程中可能发生什么事件会修改 `sepc`？）

第八节：步骤6——更新 Makefile

在 `SRCS` 中加入蹦床汇编文件 `kernel/trap/trampoline.S`。

第九节：完整改造的检查清单

完成所有步骤后，用以下清单逐项检查：

#	检查项	验证方法
1	<code>trampoline.S</code> 中的偏移量与 <code>struct trapframe</code> 严格对应	逐字段核对 <code>proc.h</code>
2	<code>kernel.ld</code> 中蹦床段页对齐	<code>objdump -h kernel.elf</code> 检查地址
3	<code>kvmminit()</code> 中映射了蹦床页到 <code>TRAMPOLINE</code>	GDB 检查 <code>walk(kernel_pagetable, TRAMPOLINE, 0)</code>
4	<code>uvmcreate()</code> 中映射了蹦床页和 <code>trapframe</code> 到用户页表	GDB 检查 <code>walk(p->pagetable, TRAMPOLINE, 0)</code>
5	<code>userinit()</code> 中 <code>epc = 0</code> (而非物理地址)	GDB: <code>p proc[0].trapframe->epc</code>
6	<code>usertrapret()</code> 中 <code>stvec</code> 指向蹦床虚拟地址	GDB: <code>info reg stvec</code>
7	<code>usertrapret()</code> 中设置了 <code>sscratch = TRAPFRAME</code>	GDB: <code>info reg sscratch</code>
8	<code>usertrapret()</code> 最终调用 <code>userret</code> 函数指针 (而非 <code>asm("sret")</code>)	代码审查
9	<code>riscv.h</code> 中有 <code>w_sscratch()</code> 和 <code>r_sscratch()</code>	编译检查
10	<code>Makefile</code> 中包含了 <code>trampoline.S</code>	编译检查

第十节：常见错误与诊断

症状	最可能的原因	排查方向
<code>sret</code> 后立即 Instruction Page Fault (scause=12)	用户页表没映射蹦床页，或 <code>stvec</code> 仍指向内核地址	检查 <code>uvmcreate()</code> 中蹦床映射；检查 <code>usertrapret()</code> 中 <code>stvec</code> 的计算

<code>sret</code> 后 Store Page Fault (scause=15)	<code>sscratch</code> 未正确设置, 或 trapframe 在用户页表中未映射	检查 <code>w_sscratch(TRAPFRAME)</code> 和 <code>uvmcreate()</code> 中的 trapframe 映射
系统死机无任何输出	页表切换后 PC 找不到代码	用 GDB <code>stepi</code> 单步追踪 <code>satp</code> 切换前后的 PC 值
寄存器值错乱 (返回用户态后行为异常)	<code>trampoline.S</code> 中偏移量不匹配	逐字段核对 <code>struct trapframe</code> 的偏移量
<code>ecall</code> 后进入 <code>sys_trap_handler</code> 而非 <code>usertrap</code>	<code>stvec</code> 没有被切换到蹦床地址	检查 <code>usertrapret()</code> 中的 <code>w_stvec()</code>
编译报 <code>undefined reference to trampoline</code>	链接脚本中缺少 <code>PROVIDE(trampoline = .)</code>	修改 <code>kernel.ld</code>
<code>.section</code> 名不匹配	<code>.section trampsec</code> 与链接脚本中的段名不一致	确保 <code>trampoline.S</code> 和 <code>kernel.ld</code> 中的名字完全相同
时钟中断消失	<code>usertrapret()</code> 没设 <code>SSTATUS_SPIE</code>	检查 <code>sstatus</code> 配置
<code>fork()</code> 后子进程崩溃	子进程页表缺少蹦床映射	确认 <code>uvmcreate()</code> 创建子页表时映射了蹦床和 trapframe

GDB 调试要点

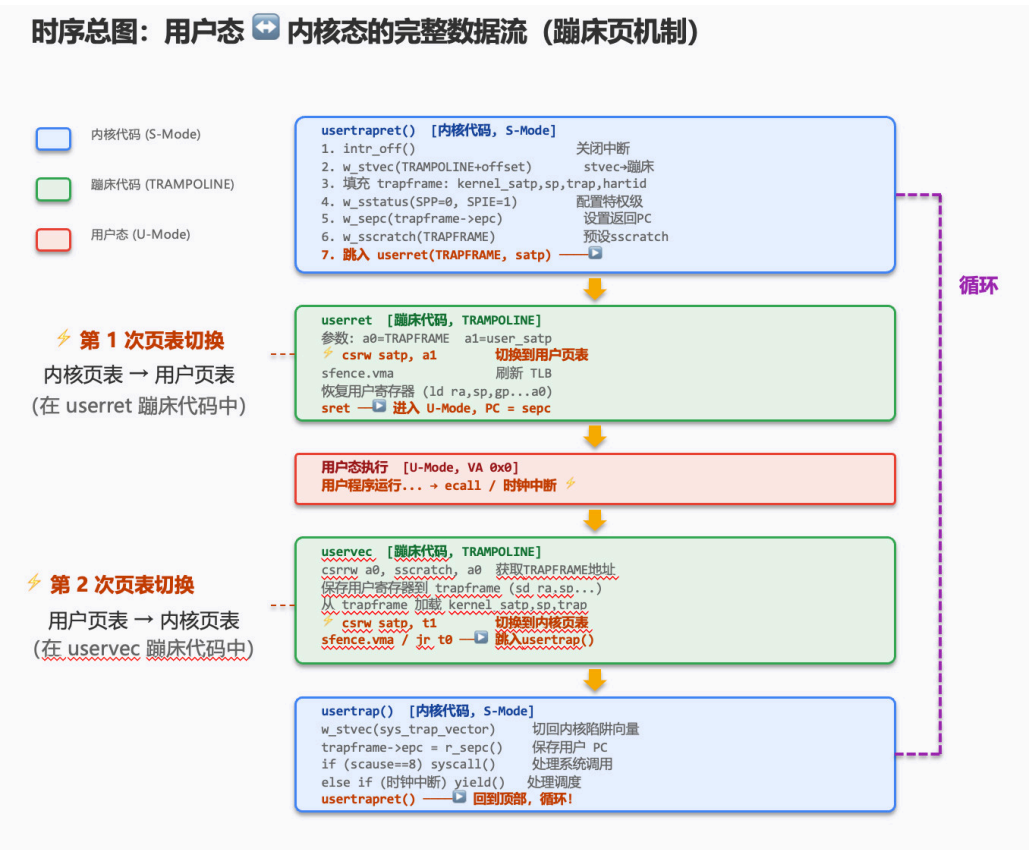
```

1  # 在 usertrapret() 返回用户态前检查关键寄存器
2  (gdb) b usertrapret
3  (gdb) c
4  # 到达断点后 :
5  (gdb) p/x p->pagetable           # 用户页表地址
6  (gdb) p/x p->trapframe->epc       # 用户 PC ( 应该是 0 )
7  (gdb) info reg satp               # 当前页表 ( 还是内核的 )
8  (gdb) info reg stvec              # 应该指向 TRAMPOLINE + offset
9
10 # 在蹦床代码中设断点
11 (gdb) b *0x3fffffff000           # TRAMPOLINE 虚拟地址 ( Sv39: MAXVA - PGSIZE )
12
13 # 单步追踪 satp 切换
14 (gdb) stepi
15 (gdb) info reg satp               # 观察 satp 何时变化

```

第十一节：时序总图——理解完整的数据流

学完以上所有内容后，请对照此图检验你对整个流程的理解是否完整：



🤔 **最终思考：** 回顾整个流程，你能数出从用户程序执行 `ecall` 到内核完成系统调用返回用户态，一共经历了几次页表切换？每次切换发生在哪里？